

Automated Tool Manager for eSim

1. Introduction

eSim is an open-source Electronic Design Automation (EDA) tool that integrates several external tools such as Ngspice, KiCad, Scilab, and Verilator for circuit design, simulation, and analysis. Each of these tools comes with its own dependencies, installation methods, and platform-specific behaviors. Manually managing these tools across different operating systems can be time-consuming and error-prone.

This project proposes an **Automated Tool Manager** for eSim that focuses on **dependency checking and a user-friendly command-line interface (CLI)**. The goal is to ensure that all required tools and dependencies are correctly installed, compatible, and easily verifiable in an OS-independent manner.

The prototype demonstrates tool installation handling using a controlled proof-of-concept approach. For selected tools such as Verilator, the system detects absence and provides OS-specific installation commands. This design can be extended to fully automated installation in future iterations.

2. Objectives

The key objectives of the Automated Tool Manager are:

- Automatically check the presence of required external tools and their dependencies.
- Detect the operating system and apply OS-specific logic internally while maintaining a unified interface.
- Provide clear feedback to the user regarding missing or incompatible dependencies.
- Offer a simple and intuitive command-line interface for interacting with the tool manager.
- Display installed tools, detected versions, and dependency status.
- Maintain logs of actions such as checks, errors, and system responses.

3. Scope of Implementation

For this prototype, the tool manager focuses on the following two deliverables:

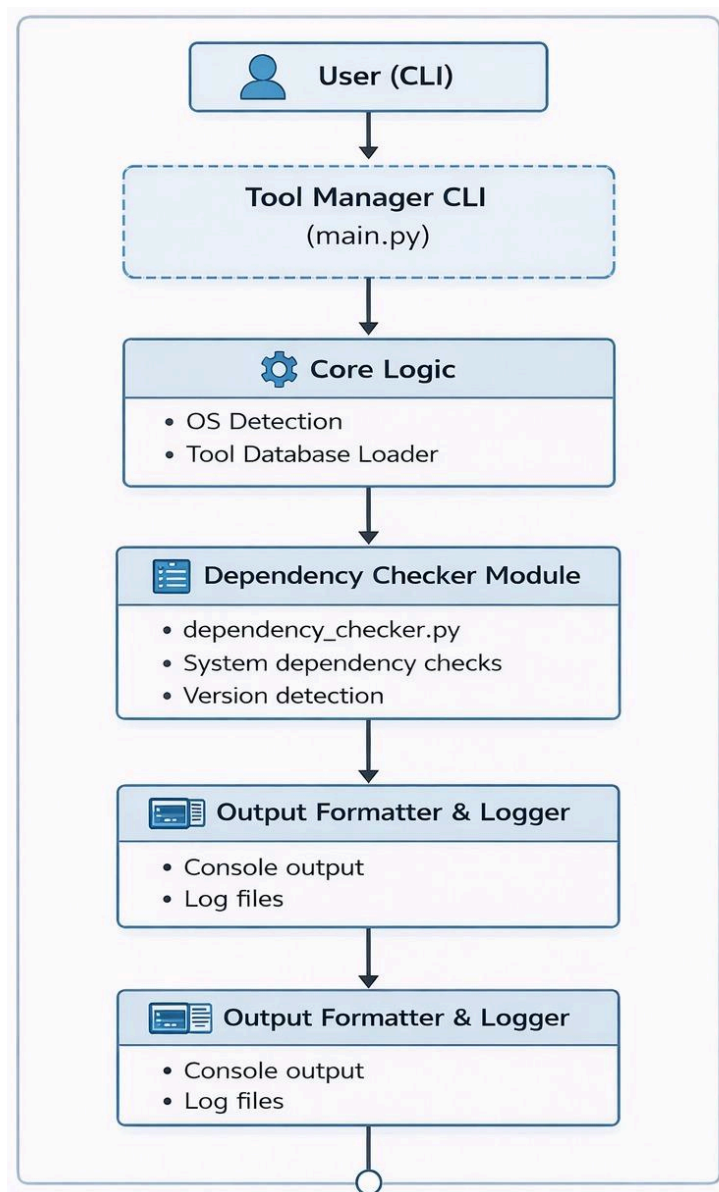
1. **Dependency Checker**
 - o Verifies required dependencies for installed tools.
 - o Handles system tools (e.g., Verilator, Scilab, Java) and Python packages.
 - o Provides OS-aware detection to avoid false positives.
2. **User Interface (CLI)**

- o Allows users to run dependency checks using simple commands.
- o Displays result in a readable tabular or structured format.
- o Logs all actions and errors for traceability.

Installation automation and update mechanisms are considered future enhancements and are included conceptually in the architecture.

4. Overall Architecture

The Automated Tool Manager follows a modular architecture where each component has a clearly defined responsibility.



5. Module Breakdown

5.1 Tool Manager CLI (main.py)

- Acts as the **entry point** for the application.
- Accepts user commands (e.g., run dependency checks).
- Invokes the dependency checker module.
- Formats and displays results to the user.

5.2 Dependency Checker (dependency_checker.py)

- Core module responsible for validating tool dependencies.
- Reads tool and dependency information from a configuration file (tool_db.json).
- Performs:
 - System command checks (using OS-aware logic).
 - Executable path checks on Windows.
 - Version extraction where applicable.
- Returns structured results indicating installation status and detected versions.

5.3 Tool Database (tool_db.json)

- Stores metadata about tools and their dependencies.
- Allows easy extension for adding new tools in the future.
- Decouples configuration from code logic.

5.4 OS Detection Utility

- Uses Python's built-in OS detection mechanisms.
- Determines whether the system is Windows, Linux, or macOS.
- Enables OS-specific dependency detection strategies internally.

5.5 Logging System

- Records actions performed by the tool manager.
- Logs dependency check results and errors.
- Helps in debugging and traceability.

6. Dependency Checking Design

The dependency checker is designed to be **OS-independent but OS-aware**:

- On **Windows**, GUI-based tools such as Scilab are detected using known executable paths (e.g., Scilab.exe).
- On **Linux/macOS**, CLI-based detection is used (e.g., scilab, scilab-cli, verilator --version).
- Java is checked separately as it is a common dependency for Scilab.

Each dependency check produces one of the following outcomes: - Installed and version detected
- Installed but version unknown - Not installed

Clear feedback is provided to the user for each case.

7. User Interface Design (CLI)

The command-line interface is designed to be minimal and user-friendly:

- Single command to run all dependency checks.
- Clear display of:
 - o Tool name
 - o Dependency name
 - o Installation status
 - o Version (if available)
- Errors and warnings are displayed in plain language.

Example output:

```
PS C:\Users\lenovo\eSimToolManager> python main.py check
Tool    Dependency    Status
ngspice python3 OK
ngspice gcc OK
kicad    python3 OK
kicad    cmake OK
verilator      verilator      OK
scilab    scilab OK
scilab    java OK
```

8. Interaction Between Components

1. The user runs the tool manager via the CLI.
2. The CLI invokes the core logic.
3. The core logic detects the operating system.
4. The dependency checker validates each required dependency using OS-specific strategies.
5. Results are collected and passed to the formatter.
6. Output is displayed to the user and logged.

9. Extensibility and Future Enhancements

The current design allows easy extension for:

- Automatic installation using package managers (apt, Chocolatey, Homebrew).

- Tool update and upgrade mechanisms.
- Graphical User Interface (GUI).
- Version compatibility enforcement.
- Integration with eSim's internal configuration system.

10. Conclusion

The Automated Tool Manager provides a modular, OS-independent solution for managing eSim's external tool dependencies. By focusing on reliable dependency checking and a clean CLI-based user interface, the system significantly reduces setup complexity and improves user experience. The current prototype serves as a strong foundation for future enhancements such as automated installation and updates.